MongoDB Cheat Sheet

Compact reference for quick exam prep — common commands, operators, aggregation, indexing, and examples.

Quick shell & connection

```
# connect (mongosh)
mongosh "mongodb://username:password@localhost:27017/mydb"

# show databases
show dbs
# switch DB
use mydb
# show collections
show collections
```

Basic CRUD (mongosh examples)

Insert

```
// single
db.users.insertOne({name: "Asha", age: 21, tags: ["student", "pref1"]})
// multiple
db.users.insertMany([{name: "Ravi"},{name: "Meera"}])
```

Find (query + projection)

```
// find all
db.users.find()
// filter and projection
db.users.find({age: {$gte: 18}}, {name: 1, _id: 0}).pretty()
// findOne
db.users.findOne({name: "Asha"})
```

Update

```
// update one
db.users.updateOne({name: "Asha"}, {$set: {age: 22}})
// update many with $inc
db.users.updateMany({status: "active"}, {$inc: {score: 10}})
// upsert (insert if not exists)
db.users.updateOne({email: "x@a.com"}, {$set: {email: "x@a.com", name: "X"}},
{upsert: true})
```

Replace

```
db.users.replaceOne({name: "Ravi"}, {name: "Ravi", age: 30})
```

Delete

```
db.users.deleteOne({name: "Meera"})
db.users.deleteMany({inactive: true})
```

Query Operators (common)

- Comparison: `$eq`, `$ne`, `$gt`, `$gte`, `$lt`, `$lte`, `$in`, `$nin`
- Logical: `$and`, `$or`, `$not`, `$nor`
- Element: `$exists`, `$type`
- Evaluation: `$regex`, `$expr`
- Array: `$all`, `$elemMatch`, `$size`

```
// find users whose tag contains "student"
db.users.find({tags: {$in: ["student"]}})
// array element match
db.products.find({reviews: {$elemMatch: {rating: {$gte: 4}}}})
```

Projection & Sorting

```
db.orders.find({status: "shipped"}, {customer:1, total:1, _id:0}).sort({total:
-1}).limit(5)
```

Update Operators (common)

- `$set`, `$unset`, `$inc`, `$mul`, `$rename`, `$min`, `$max`
- Array updates: `$push`, `$pop`, `$pull`, `$addToSet`, `$pullAll`
- `$setOnInsert`, `$currentDate`
- ArrayFilters (for nested arrays) example:

```
// increment qty for array elements matching condition
db.orders.updateOne({ _id: 1 }, { $inc: { 'items.$[elem].qty': 2 } }, {
arrayFilters: [{ 'elem.product': 'p1' }] })
```

Aggregation Pipeline (essential)

- Stages: `$match`, `$project`, `$group`, `$sort`, `$limit`, `$skip`, `$unwind`, `$lookup`, `$addFields`, `$replaceRoot`

```
// total sales per product
db.sales.aggregate([
  { $match: { status: 'complete' } },
  { $group: { _id: '$productId', totalQty: { $sum: '$quantity' }, totalRevenue:
{ $sum: { $multiply: ['$quantity', '$price'] } } } },
  { $sort: { totalRevenue: -1 } }
])
```

$lookup (join)

```
db.orders.aggregate([
  { $lookup: { from: 'customers', localField: 'custId', foreignField: '_id',
as: 'customer' } },
  { $unwind: '$customer' }
])
```

Indexes

```
// create index
db.users.createIndex({email: 1}, {unique: true})
// compound index
db.orders.createIndex({custId: 1, createdAt: -1})
// TTL index (expire after 3600 seconds)
db.sessions.createIndex({createdAt: 1}, {expireAfterSeconds: 3600})
```

```
// list indexes
db.collection.getIndexes()
// drop index
db.collection.dropIndex('email_1')
```

Tips: choose index keys for selective queries, use compound indexes in query order, avoid over-indexing.

Explain & Performance

```
db.users.find({age: {$gt: 30}}).explain('executionStats')
```

- Check `executionStats.nReturned`, `totalDocsExamined`, `indexUsed`.

Transactions (replica set required)

```
const session = db.getMongo().startSession()
session.startTransaction()
try {
  session.getDatabase('mydb').users.insertOne({name: 'T'}, {session})
  session.commitTransaction()
} catch (e) {
  session.abortTransaction()
}
session.endSession()
```

Backup & Import/Export

```
# dump
mongodump --uri="mongodb://user:pass@host:27017/mydb" --out=/backups/backup1
# restore
mongorestore --uri="mongodb://user:pass@host:27017/" /backups/backup1
# export/import (JSON)
mongoexport --db mydb --collection users --out users.json
mongoimport --db mydb --collection users --file users.json --jsonArray
```

Security basics

```
db.createUser({user: 'appUser', pwd: 'S3cret!', roles: [{role:'readWrite',
db:'mydb'}]})
```

- Enable authentication, use roles, don't expose `bindIp` to 0.0.0.0 in production.

Connection string examples

```
mongodb://user:pass@host:27017/mydb?authSource=admin&replicaSet=rs0
# DNS seedlist (Atlas)
mongodb+srv://user:pass@cluster0.abcd.mongodb.net/mydb
```

Common CLI helpers

- `db.collection.find().pretty()`
- `db.collection.countDocuments({})`
- `db.collection.stats()`
- `db.collection.aggregate([...]).itcount()`
- `db.collection.watch()` — change stream (requires replica set)

Data modeling tips

- Model for your query patterns.
- Embed for 1:many small and frequently-read data (denormalize), reference for large/independent objects.
- Keep documents < 16MB.
- Use capped collections for high-throughput logs.

Short example: Users + Orders

```
// create user
db.users.insertOne({_id: ObjectId(), name: 'A', email: 'a@x.com'})
// create order referencing user
db.orders.insertOne({userId: ObjectId('...'), items: [{product: 'p1', qty:2}],
total: 100, createdAt: new Date()})
// aggregate orders by user
db.orders.aggregate([
  { $group: { _id: '$userId', orders: { $sum: 1 }, totalSpent: { $sum:
'$total' } } },
```

```
  { $sort: { totalSpent: -1 } }
])
```

Good luck on your exam! If you want this exported as a PDF, a printable version, or a one-page summary, tell me which format and I'll prepare it.